

How to Import CSV into Neo4j

1. Prepare Your CSV File

Ensure your CSV file is properly formatted. For example, a file named `users.csv`:

```
userId,name,age,city
```

```
1,Alice,30,New York
```

```
2,Bob,25,San Francisco
```

```
3,Charlie,35,Los Angeles
```

How to Import CSV into Neo4j

2. Place the CSV File in the Import Directory

For security reasons, Neo4j only allows loading CSV files from the import directory. Copy your CSV file to the Neo4j import directory, typically located at:

- /var/lib/neo4j/import (Linux)
- C:\Program Files\Neo4j\import (Windows)

How to Import CSV into Neo4j

3. Start Neo4j

Ensure your Neo4j server is running. You can start it using the Neo4j desktop application or the command line:

```
neo4j start
```

How to Import CSV into Neo4j

4. Open Neo4j Browser

Navigate to the Neo4j browser at <http://localhost:7474> and log in with your credentials.

How to Import CSV into Neo4j

5. Load CSV Data Using Cypher

Use the `LOAD CSV` clause to import the data. Here's an example Cypher query to create nodes from the `users.csv` file:

```
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row  
  
CREATE (u:User {userId: row.userId, name: row.name, age: toInteger(row.age), city: row.city});
```

Detailed Explanation:

- `LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row`: This part of the query loads the CSV file and treats the first row as headers. Each row is represented as a map and can be accessed using the variable `row`.

- `CREATE (u:User {userId: row.userId, name: row.name, age: toInteger(row.age), city: row.city})`: This part of the query creates a new node with the label `User` for each row in the CSV file, using the data from the CSV to set the properties of the node.

How to Import CSV into Neo4j

6. Example: Importing Relationships

If you have another CSV file that represents relationships, for example `friendships.csv`:

```
userId1,userId2,since
```

```
1,2,2020
```

```
1,3,2019
```

You can load and create relationships between users like this:

```
LOAD CSV WITH HEADERS FROM 'file:///friendships.csv' AS row
```

```
MATCH (u1:User {userId: row.userId1}), (u2:User {userId: row.userId2})
```

```
CREATE (u1)-[:FRIEND {since: toInteger(row.since)}]-(u2);
```

Detailed Explanation:

- `LOAD CSV WITH HEADERS FROM 'file:///friendships.csv' AS row`: Loads the CSV file with headers.
- `MATCH (u1:User {userId: row.userId1}), (u2:User {userId: row.userId2})`: Finds the user nodes corresponding to `userId1` and `userId2`.
- `CREATE (u1)-[:FRIEND {since: toInteger(row.since)}]-(u2)`: Creates a `FRIEND` relationship between the two user nodes with a `since` property.